정규교과 2/4

머신러닝 모델링 및 성능평가

Day 6 · 선형회귀 · 로지스틱회귀 → KNN → 공정한 모델 비교

한로 강민준

어제 우리가 배운 것

문제 정의: 예측 시점에 존재하는 입력 X와 정답 y를 구분했다.

분할: train 60 · valid 20 · test 20의 역할과 ID를 고정했다.

Pipeline: train에서만 전처리를 학습하고 새 입력에 재사용했다.

오늘의 확장: Dummy 기준선 자리에 선형회귀 · 로지스틱회귀 · KNN을 연결한다.

공식 참고: scikit-learn · Pipeline / Common pitfalls

3분 확인 퀴즈 · Day 5 복습

Q1. valid에서 imputer.fit_transform을 다시 해도 될까?

정답·해설 안 된다. train에서 학습한 대표값으로 valid를 transform한다.

Q2. 새 모델을 비교할 때 데이터 분할도 매번 바꿀까?

정답·해설 아니다. 같은 ID·같은 입력·같은 지표를 사용해야 비교할 수 있다.

Q3. test로 가장 좋은 K를 골라도 될까?

정답·해설 안 된다. valid로 선택하고 test는 최종 확인까지 남겨 둔다.

공식 참고: scikit-learn · Common pitfalls / Cross-validation

오늘의 목표와 8교시 운영

교시	핵심 내용	설명·시연	실습·리뷰
1	선형회귀 원리 · 첫 학습	30분	20분
2	회귀 지표 · 기준선 · 오류 해석	25분	25분
3	로지스틱회귀 · 확률 · 분류 오류	30분	20분
4	KNN 분류 · 회귀 · 거리 · 가중치	25분	25분
5	스케일링 · 열별 전처리 · Pipeline	25분	25분
6	K·하이퍼파라미터 비교 · 선택	25분	25분
7	회귀·분류 비교표 · 프로젝트 통합	25분	25분
8	검증 · 지표 선택 근거 · 시연	15분	35분

50분 × 8교시 = 400분 · 설명/시연 200분 + 실습/리뷰 200분 · 교시 사이 10분 휴식

오늘 배우는 범위와 다음 수업의 경계

오늘 · 10/26

선형·로지스틱회귀와 KNN의 예측 원리

스케일링과 K 후보의 valid 비교

두 문제의 비교표와 선택 근거 작성

다음 수업

10/27: 트리 · Random Forest · XGBoost

10/28: 지표 심화 · 교차검증 · 최종 평가

10/29~30: ANN · 학습과 최적화

오늘은 valid로 후보를 비교한다. 최종 test 평가는 모델·평가 정책을 확정한 뒤 수행한다.

공식 참고: scikit-learn · Cross-validation

환경 점검 · 기존 가상환경을 유지한다

```
python --version
python -m pip show numpy pandas scikit-learn
python -c "import sklearn; print(sklearn.__version__)"

# Windows PowerShell: 프로젝트 인터프리터 확인
.venv\Scripts\python.exe -m pip check

# 새 환경은 강사가 검증한 requirements로 준비
# 수업 직전에 무조건 최신 버전으로 올리지 않는다.
```

SHELL | 프로젝트 가상환경에서 실행

확인할 사항

VS Code 인터프리터
프로젝트의 .venv 선택

pip check
의존성 충돌 확인

아래는 자료 검증 환경

자료 검증 환경

```
Python 3.13.5
NumPy 2.3.5
pandas 2.2.3
scikit-learn 1.8.0
```

같은 요청으로 두 가지 질문을 만든다

구분	회귀	분류
질문	실행에 몇 ms가 걸릴까?	실행이 실패할까?
정답 y	elapsed_ms · 연속적인 수치	failed · 0=성공, 1=실패
입력 X	실행 전에 알고 있는 4개 열	같은 4개 열
모델 후보	LinearRegression · KNN 회귀	LogisticRegression · KNN 분류
주 지표(수업 가정)	큰 오차를 중시하는 RMSE	실패 클래스의 Precision·Recall을 함께 보는 F1

elapsed_ms는 회귀의 정답이 될 수 있다. 그러나 실행 전 예측의 입력 X로 쓰면 누수다.

공식 참고: scikit-learn · Common pitfalls

공통 데이터 · Day 5의 100건을 그대로 사용

구성	열 / 정책	의미
수치형 입력	input_chars · allowed_tools	글자 수 · 허용 도구 수
범주형 입력	task_type · priority	작업 유형 · 우선순위
학습에 넣지 않을 열	request_id · status · retry_count	식별자 또는 실행 후 정보
회귀·분류 정답	elapsed_ms / failed	두 문제별로 y를 별도 선택
자료의 성격	수업용 합성 데이터 · 일부 결측 포함	실서비스의 성능 근거로 사용하지 않음

작은 원리 예제와 100건 프로젝트 자료를 구분한다. 표·그래프에는 어떤 자료인지 표시한다.

전날의 분할 ID를 두 문제에 함께 적용

```
yr_train = data.loc[X_train.index, "elapsed_ms"]
yr_valid = data.loc[X_valid.index, "elapsed_ms"]
print("분할:", len(X_train), len(X_valid), len(X_test))
print("valid 라벨:", y_valid.value_counts().sort_index().tolist())
print("train 중앙값:", X_train[NUM].median().tolist())
print("회귀 y 단위: ms")
print("회귀 정렬:", yr_train.index.equals(X_train.index))
```

E05 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

X_train / X_valid
이미 정한 입력과 행 순서

.loc[ID, 열]
회귀 정답도 같은 ID로 정렬

회귀는 연속 y로 층화하지 않음

실행 결과 · 요약

```
train / valid / test
60 / 20 / 20
valid: 성공 14 · 실패 6
회귀 정렬: True
```

공식 참고: scikit-learn · train_test_split / Common pitfalls

선형회귀 · 입력의 가중합으로 숫자를 예측

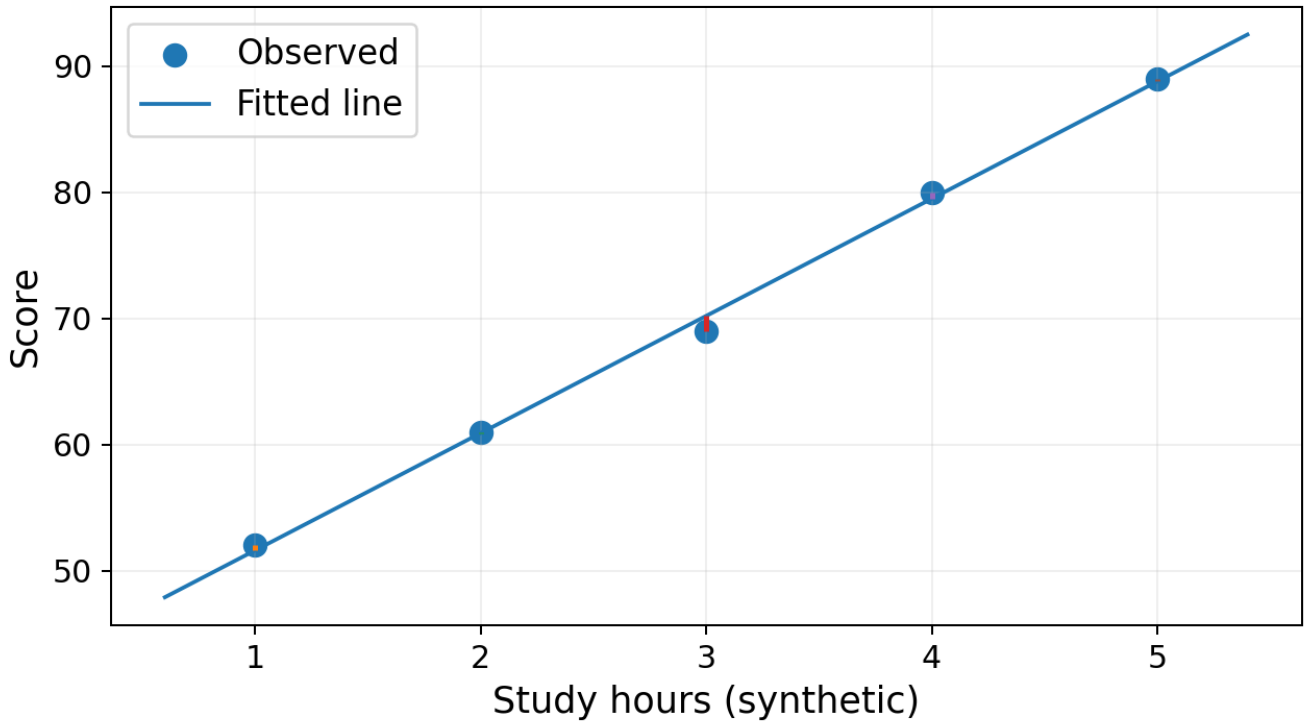
$$\text{예측 } \hat{y} = w \times x + b$$

x	입력 특성 · 예: 공부시간
w	기울기 · x가 1 늘 때 예측 변화량
b	절편 · x=0일 때의 예측값

fit: 학습 데이터의 잔차 제곱합을 작게 만드는 w와 b를 찾는다.

공식 참고: scikit-learn · LinearRegression / Linear models

관측값과 예측 직선 사이의 차이가 잔차다



잔차 = 실제값 − 예측값

점: 관측한 5개의 가상 점수

선: 학습으로 찾은 예측 규칙

선과 점의 수직 차이를 줄인다.

학습 오차가 작아도 새 자료에 잘 맞는지는 별도 평가다.

원리용 자료: 공부시간 1~5시간 / 점수 [52, 61, 69, 80, 89]

공식 참고: scikit-learn · LinearRegression

LinearRegression · 학습하고 새 값을 예측

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([52, 61, 69, 80, 89])
model = LinearRegression()
model.fit(X, y)
print("기울기:", model.coef_.round(1))
print("절편:", round(model.intercept_, 1))
print("6시간:", model.predict([[6]]).round(1))
```

E01 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

fit(X, y)
계수와 절편 학습

coef_ / intercept_
학습된 계수 / 절편

predict([[6]])
1행 × 1열의 새 입력

실행 결과 · 요약

```
기울기: [9.3]
절편: 42.3
6시간: [98.1]
```

공식 참고: scikit-learn · LinearRegression

계수와 예측을 해석할 때 주의할 점

입력이 여러 개면 $\hat{y} = w_1x_1 + w_2x_2 + \dots + b$로 확장한다.

계수는 다른 입력을 고정했을 때의 변화다. 원인·결과의 증명이 아니다.

서로 단위가 다른 계수의 크기만으로 중요도를 비교하지 않는다.

관측 범위 밖으로 예측하는 외삽은 별도 위험으로 기록한다.

공식 참고: scikit-learn · Linear models / LinearRegression

실습 01 · 선형회귀로 숫자 예측

기본필수

E01의 X·y를 확인하고 학습한다. 계수 9.3, 절편 42.3,
6시간 예측 98.1을 재현하고 X의 2차원 형태를 설명한다.

응용

입력 2개인 원리 예제(E02)를 실행한다. 계수와 절편을
수식으로 적고 [400, 2]의 900ms 예측을 직접 계산한다.

심화

한 점을 이상치로 바꾸어 계수·예측의 변화를 비교한다.
외삽 값 하나를 추가하고 결과를 신뢰할 수 없는 이유를 적는다.

완료 기준: 입력 형태 + 계수·절편 + 예측 1건 + 학습과 예측의 차이

회귀 평가 · 학습과 검증의 역할을 분리

train

전처리 학습
모델 계수 학습

valid

같은 규칙으로 예측
오차를 계산·비교

test

후보 선택에서 제외
최종 평가까지 보관

좋은 학습 점수보다, 같은 valid에서 기준선보다 무엇이 나아졌는지를 확인한다.

공식 참고: scikit-learn · Common pitfalls / Cross-validation

회귀 지표 · 크기·단위·비교 기준을 함께 읽기

지표	계산의 핵심	해석
MAE	|실제 − 예측|의 평균	정답과 같은 단위 · 작을수록 좋음
MSE	(실제 − 예측)²의 평균	단위가 제곱됨 · 큰 오차에 더 큰 벌점
RMSE	MSE의 제곱근	정답과 같은 단위 · 큰 오차에 민감
R^2	1 − 잔차제곱합 / 전체제곱합	최대 1 · 음수 가능 · 오류 비율이 아님

소요시간을 ms로 예측하면 MAE·RMSE도 ms다. “정확도 90%”로 바꾸어 말하지 않는다.

공식 참고: scikit-learn · mean_absolute_error / root_mean_squared_error / r2_score

작은 배열로 MAE·RMSE·R² 확인

```
from sklearn.metrics import (mean_absolute_error,
    root_mean_squared_error, r2_score)
y_true = [100, 200, 300]
y_pred = [120, 180, 360]
mae = mean_absolute_error(y_true, y_pred)
rmse = root_mean_squared_error(y_true, y_pred)
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R2:", round(r2_score(y_true, y_pred), 2))
```

E03 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

y_true / y_pred
순서가 맞는 실제값 / 예측값

mean_absolute_error
절대오차 평균

root_mean_squared_error
제곱오차의 평균에 제곱근

실행 결과 · 요약

```
MAE: 33.33
RMSE: 38.3
R2: 0.78
```

공식 참고: scikit-learn · mean_absolute_error / root_mean_squared_error / r2_score

지표가 달라지면 선택도 달라질 수 있다

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import root_mean_squared_error
y = [100, 200, 300]
predictions = {"A": [100, 200, 400],
               "B": [140, 240, 340]}
for name, pred in predictions.items():
    mae = mean_absolute_error(y, pred)
    rmse = root_mean_squared_error(y, pred)
    print(f"{name}: MAE={mae:.2f}, RMSE={rmse:.2f}")
```

E04 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

모델 A의 오차
[0, 0, 100]

모델 B의 오차
[40, 40, 40]

큰 실패 1건을 더 피해야 한다면
RMSE 관점이 필요하다.

실행 결과 · 요약

```
A: MAE=33.33, RMSE=57.74
B: MAE=40.00, RMSE=40.00
```

공식 참고: scikit-learn · mean_absolute_error / root_mean_squared_error

R^2가 음수라는 말의 정확한 뜻

R^2의 기준은 평가 중인 y의 평균값을 항상 예측하는 경우다.

그 기준보다 잔차 제곱합이 크면 R^2는 0보다 작아질 수 있다.

DummyRegressor는 train 평균을 배운다. valid R^2가 꼭 0은 아니다.

상수인 y·극히 적은 표본에서는 R^2의 특수 동작을 따로 확인한다.

valid 평균은 R^2의 계산 기준일 뿐, 실서비스에 쓸 학습 규칙으로 미리 알고 있는 값이 아니다.

공식 참고: scikit-learn · r2_score / DummyRegressor

공통 자료 · 전처리 + 선형회귀로 20건 예측

```
reg = build(LinearRegression())
reg.fit(X_train, yr_train)
pred = reg.predict(X_valid)
mae = mean_absolute_error(yr_valid, pred)
rmse = root_mean_squared_error(yr_valid, pred)
print("예측 수:", len(pred))
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R2:", round(r2_score(yr_valid, pred), 3))
```

E06 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

build(모델)
전날의 열별 전처리와 연결

yr_train / yr_valid
elapsed_ms 정답

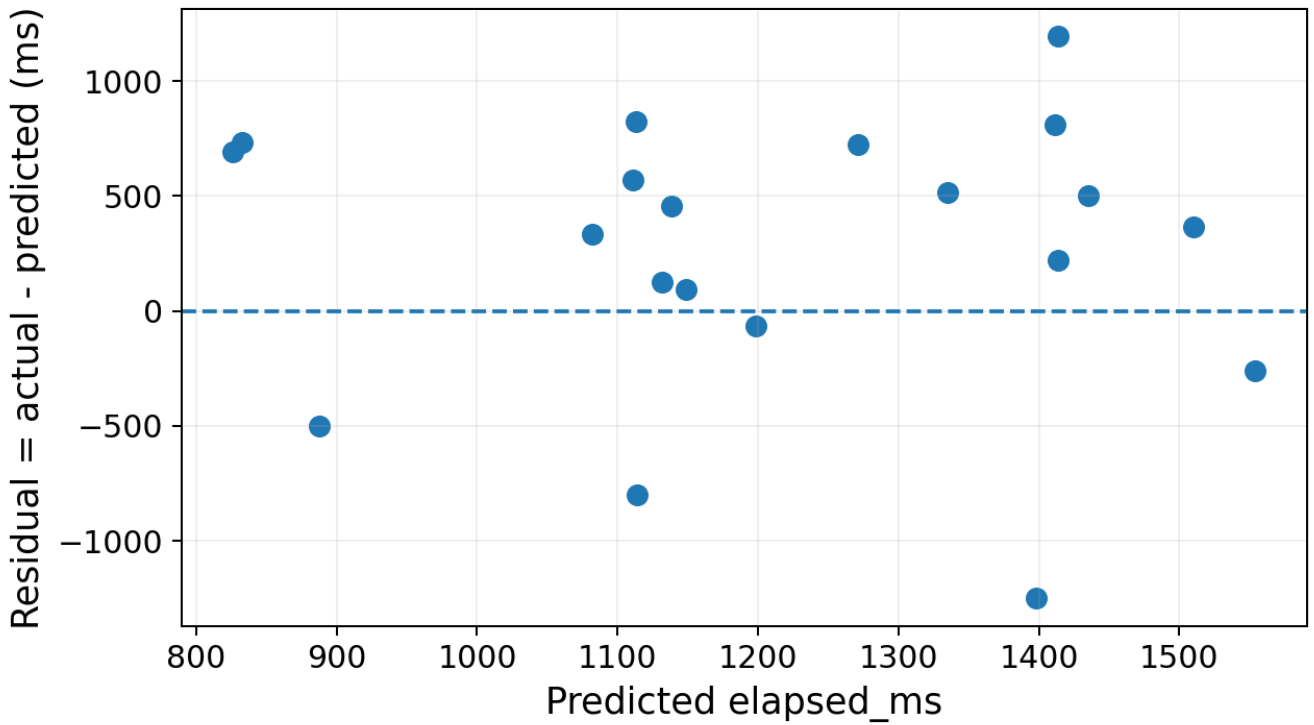
fit은 train에만
valid는 predict만 수행

실행 결과 · 요약

```
예측 수: 20
MAE: 551.8
RMSE: 640.0
R2: -0.097
```

공식 참고: scikit-learn · LinearRegression / Pipeline

평균 점수 뒤에 숨은 큰 오류를 본다



큰 잔차를 설명할 수 있는가?

세로축이 0이면 실제와 예측이 같다.

양수: 실제가 더 오래 걸렸다.

음수: 실제보다 오래 걸린다고 예측했다.

큰 오류를 삭제하지 말고 입력·단위·누수 여부를 먼저 확인한다.

Day5 합성 요청 · valid 20건 · 선형회귀 · 잔차 = 실제 elapsed_ms − 예측값

공식 참고: scikit-learn · LinearRegression / root_mean_squared_error

3분 퀴즈 · 회귀 결과 읽기

Q1. RMSE 640의 단위는 무엇인가?

정답·해설 이 데이터에서는 ms다. 정답 elapsed_ms의 단위를 따른다.

Q2. R^2가 음수이면 코드가 반드시 잘못되었을까?

정답·해설 아니다. 평가 자료 평균을 예측하는 기준보다 제곱오차가 클 수 있다.

Q3. MAE와 RMSE가 다른 모델을 선택하면?

정답·해설 업무의 오류 비용에 따라 미리 정한 주 지표로 선택하고 둘 다 기록한다.

공식 참고: scikit-learn · root_mean_squared_error / r2_score

실습 02 · 회귀 오차와 선택 근거

기본필수

E03·E04를 실행하고 MAE·RMSE의 단위와 A/B 선택 차이를 설명한다. 공통 자료 선형회귀의 valid 결과를 기록한다.

응용

DummyRegressor(mean)와 선형회귀를 같은 valid에서 비교한다. 큰 오류 3건의 실제값·예측값·잔차를 정리한다.

심화

MAE와 RMSE의 선택이 다른 사례를 만들고 업무 비용을 제안한다. 이상치 제거 전후를 비교하되 valid를 본 뒤 임의 삭제하지 않는다.

완료 기준: 회귀 지표표 + 단위 + 큰 오류 사례 + 선택 지표의 이유

로지스틱회귀 · 이름에 회귀가 있어도 분류 모델

목적: 입력으로 실패 클래스(1)의 가능성을 추정한다.

선형 점수 z를 0~1 사이 값으로 바꾼 뒤 판단 규칙을 적용한다.

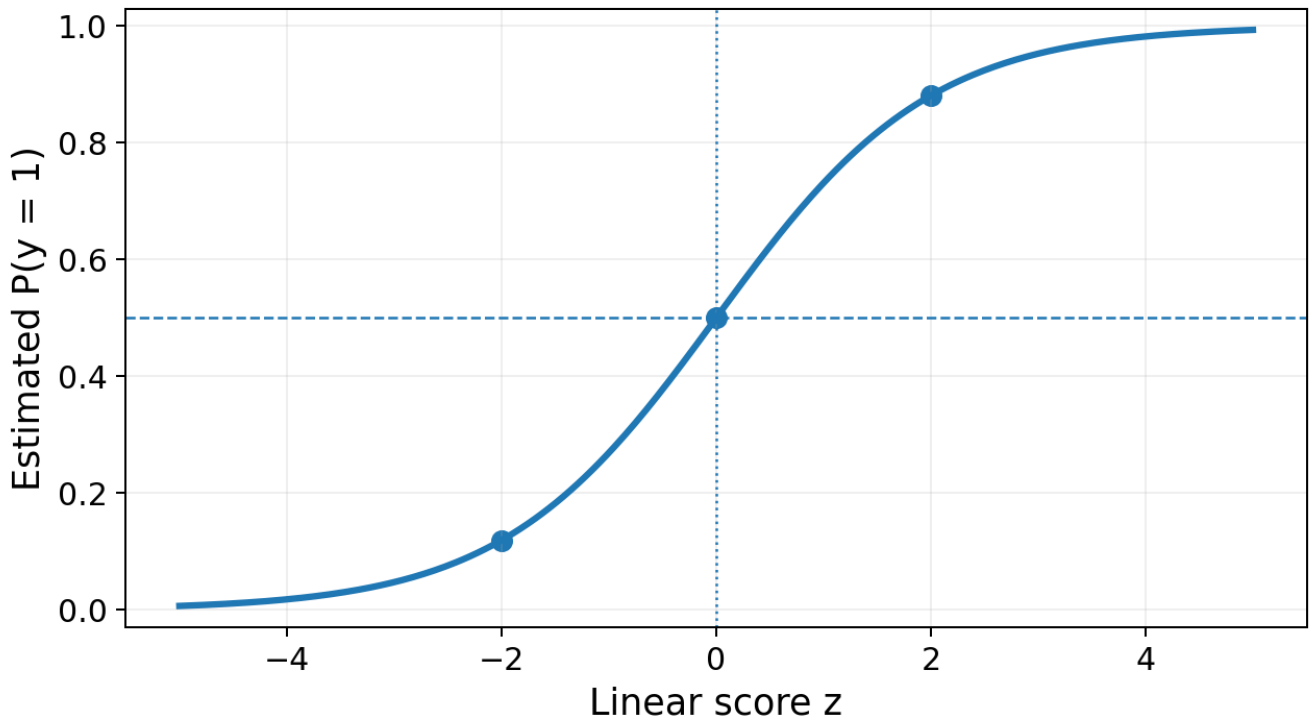
출력 확률과 최종 클래스는 서로 다른 정보다.

기본 구현에는 계수가 지나치게 커지는 것을 억제하는 규제가 있다.

실무 예: 실패 위험·이탈 가능성·스팸 여부. 이 수업에서는 성공 0 / 실패 1로 고정한다.

공식 참고: scikit-learn · LogisticRegression / Linear models

선형 점수 z → 시그모이드 → 클래스 판단



p = 1 / (1 + exp(−z))

$z = w_1x_1 + \dots + b$

z가 커질수록 실패 확률이 커진다.

z=0이면 p=0.5다.

확률 0.8은 정답을 80% 보장한다는 뜻이 아니다.

모형이 추정한 확률은 별도 검증이 필요하다. 확률 보정은 이후 학습 주제다.

공식 참고: scikit-learn · Linear models / LogisticRegression

점수·확률·판단 규칙을 직접 계산

```
import numpy as np
z = np.array([-2.0, 0.0, 2.0])
p = 1 / (1 + np.exp(-z))
print("점수 z:", z.tolist())
print("확률 p:", p.round(3).tolist())
print("규칙 p>=0.5:", (p >= 0.5).astype(int).tolist())
```

E07 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

np.exp()
자연상수 e의 거듭제곱

(p >= 0.5)
직접 정한 임계값 규칙

astype(int)
False/True를 0/1로 변환

실행 결과 · 요약

```
z: [-2.0, 0.0, 2.0]
p: [0.119, 0.5, 0.881]
규칙 결과: [0, 1, 1]
```

공식 참고: scikit-learn · Linear models / Decision threshold

학습할 때 줄이는 손실과 평가 지표는 다르다

로지스틱회귀의 학습

정답 클래스에 낮은 확률을 주면 손실이 커진다.

실제 y=1일 때 p=0.9보다 p=0.1이 더 큰 손실이다.

로그 손실과 규제 항을 고려해 계수를 찾는다.

수업의 분류 평가

모델 선택은 valid의 F1을 주 지표로 한다.

Precision·Recall·FP·FN도 함께 보고한다.

학습이 F1을 직접 최적화한다고 설명하지 않는다.

max_iter는 최적화 반복의 상한이다. 큰 값을 넣는 것만으로 일반화 성능이 보장되지 않는다.

공식 참고: scikit-learn · log_loss / LogisticRegression

predict_proba와 predict를 구분한다

```
clf = build(LogisticRegression(C=1.0, max_iter=1000))
clf.fit(X_train, y_train)
classes = clf.named_steps["model"].classes_
pos = list(classes).index(1)
p = clf.predict_proba(X_valid)[:, pos]
pred = clf.predict(X_valid)
print("classes:", classes.tolist())
print("앞 4개 p:", p[:4].round(3).tolist())
print("앞 4개 예측:", pred[:4].tolist())
```

E08 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

C=1.0
규제 강도의 역수

max_iter=1000
최적화 반복 상한

classes_ / [:, pos]
클래스 순서 / 실패 확률 열

실행 결과 · 요약

```
classes: [0, 1]
p: [0.141, 0.185,
    0.601, 0.745]
예측: [0, 0, 1, 1]
```

공식 참고: scikit-learn · LogisticRegression

혼동행렬

		실제 정답	
		True	False
분류 결과	True	True Positive	False Positive
	False	False Negative	True Negative

공식 참고: scikit-learn · confusion_matrix

오류 두 종류를 분리해서 읽는다

	예측 성공 (0)	예측 실패 (1)
실제 성공 (0)	TN · 10	FP · 4
실제 실패 (1)	FN · 4	TP · 2

실패를 양성(1)으로 정한다

FP · 성공을 실패로 예측
불필요한 점검·재처리

FN · 실패를 성공으로 예측
문제를 놓치는 비용

행=실제 · 열=예측 · 0=성공 · 1=실패 · valid 20건 중 실제 실패 6건

공식 참고: scikit-learn · confusion_matrix

분류 지표 · 분모가 서로 다르다

지표	계산	E09 결과
Accuracy	(TP + TN) / 전체 수	12 / 20 = 0.600
Precision	TP / (TP + FP)	2 / 6 = 0.333
Recall	TP / (TP + FN)	2 / 6 = 0.333
F1	2TP / (2TP + FP + FN)	4 / 12 = 0.333

F1은 TN을 사용하지 않는다. 어떤 오류가 중요한지 설명하지 않으면 높은 F1만으로 충분하지 않다.

공식 참고: scikit-learn · precision_score / recall_score / f1_score

혼동행렬과 분류 지표를 코드로 확인

```
clf = build(LogisticRegression(C=1.0, max_iter=1000))
clf.fit(X_train, y_train)
pred = clf.predict(X_valid)
cm = confusion_matrix(y_valid, pred, labels=[0, 1])
print(cm)
print("Accuracy:", round(accuracy_score(y_valid, pred), 3))
print("F1:", round(f1_score(y_valid, pred), 3))
```

E09 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

labels=[0, 1]
행·열의 클래스 순서 고정

confusion_matrix
실제값·예측값 조합의 개수

f1_score
기본 양성 클래스는 1

실행 결과 · 요약

```
[[10  4]
 [ 4  2]]
Accuracy: 0.6
F1: 0.333
```

공식 참고: scikit-learn · confusion_matrix / f1_score

임계값을 낮추면 놓침과 오탐이 바뀐다

```
import numpy as np
from sklearn.metrics import confusion_matrix
truth = np.array([0, 0, 0, 1, 1, 1])
p = np.array([0.10, 0.40, 0.70, 0.35, 0.60, 0.90])
for threshold in [0.5, 0.3]:
    pred = (p >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(
        truth, pred, labels=[0, 1]).ravel()
    print(f"t={threshold}: FP={fp}, FN={fn}, TP={tp}")
```

E10 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

threshold
확률을 행동으로 바꾸는 기준

낮추면 양성 예측이 늘 수 있음
FN 감소 / FP 증가 가능

선택은 valid에서만

실행 결과 · 요약

```
t=0.5: FP=1, FN=1, TP=2
t=0.3: FP=2, FN=0, TP=3
```

공식 참고: scikit-learn · Decision threshold / confusion_matrix

3분 퀴즈 · 실패 예측의 의미

Q1. 모든 요청을 성공으로 예측하면 valid Accuracy는?

정답·해설 14/20=0.70이지만 실패 6건을 하나도 찾지 못한다.

Q2. predict_proba의 두 번째 열은 항상 내가 원하는 클래스일까?

정답·해설 classes_를 먼저 확인해야 한다. 여기서는 실패 1의 열을 선택한다.

Q3. 임계값을 낮추면 손실 없이 성능만 좋아질까?

정답·해설 아니다. 놓침을 줄이는 대신 오탐이 늘 수 있다.

공식 참고: scikit-learn · LogisticRegression / Decision threshold / DummyClassifier

실습 03 · 실패 확률과 오류 분석

기본필수

E08·E09를 실행한다. 확률과 클래스의 차이를 설명하고
TN·FP·FN·TP = 10·4·4·2를 해석한다.

응용

가상 확률 E10의 임계값 0.5와 0.3을 비교한다.
두 오류 중 어떤 비용이 더 큰 업무인지 한 가지 예를 적는다.

심화

FP 1건과 FN 1건을 ID·입력과 함께 분석한다.
C 또는 임계값을 바꾸기 전에 비교할 후보와 지표를 먼저 기록한다.

완료 기준: 혼동행렬 + 실패 클래스 지표 + 오류 비용 설명 + test 미사용

KNN · 가까운 학습 사례를 참고해 예측

새 입력과 학습 자료 사이의 거리를 계산한다.

가장 가까운 K개 이웃을 찾는다.

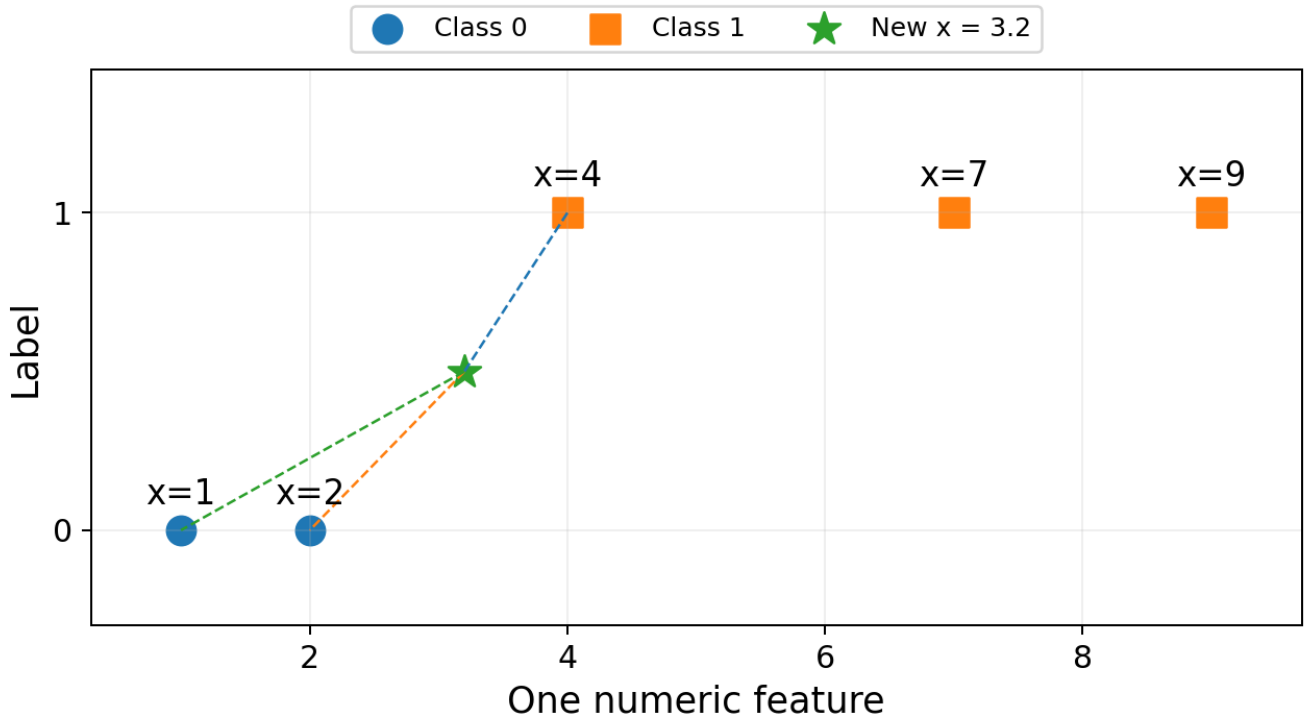
분류는 이웃의 클래스 투표, 회귀는 이웃 정답의 평균을 사용한다.

입력의 단위·특성 선택·거리 계산 방식이 이웃을 바꾼다.

KNN = K-최근접 이웃. fit에서 자료를 기억하고, predict에서 가까운 사례를 탐색한다.

공식 참고: scikit-learn · Nearest neighbors

새 값 3.2에서 가까운 세 이웃을 찾는다



K=3 · 이웃은 x=4, 2, 1

거리: 0.8, 1.2, 2.2

이웃 라벨: 1, 0, 0

균일 투표 결과: 클래스 0

정답 y는 거리 계산에 넣지 않는다.

원리용 1차원 입력 [1, 2, 4, 7, 9] · 실제 다차원에서는 모든 선택 특성으로 거리를 계산한다.

공식 참고: scikit-learn · KNeighborsClassifier

kneighbors로 선택된 이웃을 확인

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
X = np.array([[1], [2], [4], [7], [9]])
y = np.array([0, 0, 1, 1, 1])
knn = KNeighborsClassifier(n_neighbors=3).fit(X, y)
distance, index = knn.kneighbors([[3.2]])
print("가까운 x:", X[index[0], 0].tolist())
print("거리:", distance[0].round(1).tolist())
print("예측:", knn.predict([[3.2]]).tolist())
```

E11 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

n_neighbors=3
참고할 이웃 수

kneighbors()
거리와 학습 행 인덱스 반환

X[index[0], 0]
선택된 이웃의 입력값

실행 결과 · 요약

```
가까운 x: [4, 2, 1]
거리: [0.8, 1.2, 2.2]
예측: [0]
```

공식 참고: scikit-learn · KNeighborsClassifier / Nearest neighbors

KNN 회귀 · 투표 대신 숫자를 평균

```
import numpy as np
from sklearn.neighbors import KNeighborsRegressor
X = np.array([[1], [2], [4], [7], [9]])
y = np.array([10, 20, 40, 70, 90])
knn = KNeighborsRegressor(n_neighbors=3).fit(X, y)
print("예측:", knn.predict([[3.2]]).round(2).tolist())
weighted = KNeighborsRegressor(
    n_neighbors=3, weights="distance").fit(X, y)
print("거리 가중:", weighted.predict([[3.2]]).round(2).tolist())
```

E12 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

이웃 정답: 40 · 20 · 10
균일 평균 = 70 / 3

weights="distance"
가까운 이웃에 큰 가중치

정답 단위가 예측 단위가 됨

실행 결과 · 요약

```
예측: [23.33]
거리 가중: [28.06]
```

공식 참고: scikit-learn · KNeighborsRegressor / Nearest neighbors

KNN의 주요 설정과 실무상 한계

항목	설정 / 의미	주의할 점
K	n_neighbors · 이웃 수	train 행 수 이하여야 함
가중치	uniform / distance	거리 가중이 항상 더 낫지는 않음
거리	p=2: 유클리드 / p=1: 맨해튼	기본 metric=minkowski일 때
자료 규모	학습 사례를 저장하고 이웃 탐색	자료가 커지면 메모리·예측 비용 확인
특성 수	선택한 모든 특성이 거리에 참여	불필요한 특성·단위·범주 가중에 주의

공식 참고: scikit-learn · KNeighborsClassifier / KNeighborsRegressor / Nearest neighbors

관측 범위 밖에서 두 모델은 다르게 움직인다

```
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
X = [[1], [2], [3], [4]]
y = [10, 20, 30, 40]
linear = LinearRegression().fit(X, y)
knn = KNeighborsRegressor(n_neighbors=2).fit(X, y)
print("x=8 선형:", linear.predict([[8]]).round(1).tolist())
print("x=8 KNN:", knn.predict([[8]]).round(1).tolist())
```

E27 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

선형회귀
찾은 직선을 범위 밖까지 연장

KNN 회귀
학습 정답의 가중 평균

어느 쪽도 외삽 정답을 보장하지 않음

실행 결과 · 요약

```
x=8 선형: [80.0]
x=8 KNN: [35.0]
```

공식 참고: scikit-learn · LinearRegression / KNeighborsRegressor

3분 퀴즈 · KNN의 판단 근거

Q1. KNN 분류와 회귀에서 달라지는 마지막 단계는?

정답·해설 분류는 투표, 회귀는 평균이다. 이웃을 찾는 과정은 비슷하다.

Q2. K=1의 train 점수가 높으면 좋은 모델인가?

정답·해설 자기 자신을 이웃으로 포함한 결과일 수 있다. valid로 확인해야 한다.

Q3. 100개 특성 중 단위가 큰 하나가 있으면?

정답·해설 거리에서 과도한 영향을 줄 수 있다. 다음 교시에서 스케일을 비교한다.

공식 참고: scikit-learn · Nearest neighbors

실습 04 · KNN 회귀·분류를 직접 비교

기본필수

E11·E12의 이웃 3개를 직접 계산한다. 분류 결과 0과
회귀 결과 23.33을 코드 출력과 대조한다.

응용

weights를 uniform에서 distance로 바꾸고 회귀 결과 28.06을
확인한다. 예측이 바뀐 이유를 이웃 거리로 설명한다.

심화

p=1과 p=2가 다른 이웃을 선택하는 2차원 사례를 만든다.
K>학습 행 수 오류와 거리 동률 처리의 영향을 점검한다.

완료 기준: 이웃 입력·거리·정답 표 + 분류/회귀 계산 + 설정의 의미

스케일이 다르면 “가깝다”의 뜻도 달라진다

입력	글자 수	허용 도구 수	클래스
학습 A	1000	1	0
학습 B	1400	4	1
새 요청 Q	1100	4	?

원 단위: Q는 A에 가깝다. train 기준 표준화 후: 도구 수 차이가 반영되어 B에 가깝다.

공식 참고: scikit-learn · StandardScaler / Nearest neighbors

같은 데이터라도 단위 변환이 이웃을 바꾼다

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
X = np.array([[1000, 1], [1400, 4]])
y = np.array([0, 1])
q = np.array([[1100, 4]])
raw = KNeighborsClassifier(n_neighbors=1).fit(X, y)
s = StandardScaler().fit(X)
scaled = KNeighborsClassifier(n_neighbors=1)
scaled.fit(s.transform(X), y)
print("원 단위:", raw.predict(q).tolist())
print("표준화:", scaled.predict(s.transform(q)).tolist())
```

E13 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

StandardScaler.fit(X)
train 평균·표준편차 저장

transform(q)
같은 규칙으로 새 입력 변환

두 경우 모두 K=1로 고정

실행 결과 · 요약

```
원 단위: [0]
표준화: [1]
```

공식 참고: scikit-learn · StandardScaler / Nearest neighbors

정규화라는 말을 네 가지 의미로 섞지 않는다

용어	어떤 값을 바꾸는가?	대표 예
표준화	각 특성에서 평균을 빼고 표준편차로 나눔	StandardScaler
최소–최대 스케일링	각 특성의 train 최소·최대로 범위를 변환	MinMaxScaler
벡터 정규화	각 행 벡터의 길이를 1로 맞춤	Normalizer
규제 · Regularization	큰 계수 등에 벌점을 주어 학습을 제어	로지스틱회귀의 C

오늘의 필수 비교는 특성 스케일링이다. 표준화가 데이터 분포를 정규분포로 만드는 것은 아니다.

공식 참고: scikit-learn · Preprocessing / Normalizer / LogisticRegression

표준화·Min-Max · 새 값의 범위 확인

```
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler
X = np.array([[100], [200], [300]])
a = StandardScaler().fit(X)
b = MinMaxScaler().fit(X)
print("표준화:", a.transform(X).ravel().round(3).tolist())
print("MinMax:", b.transform(X).ravel().tolist())
print("새 값 500:", b.transform([[500]]).ravel().tolist())
```

E14 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

표준화
train 평균 0 · 분산 1 기준

MinMaxScaler
train에서 기본 범위 0~1

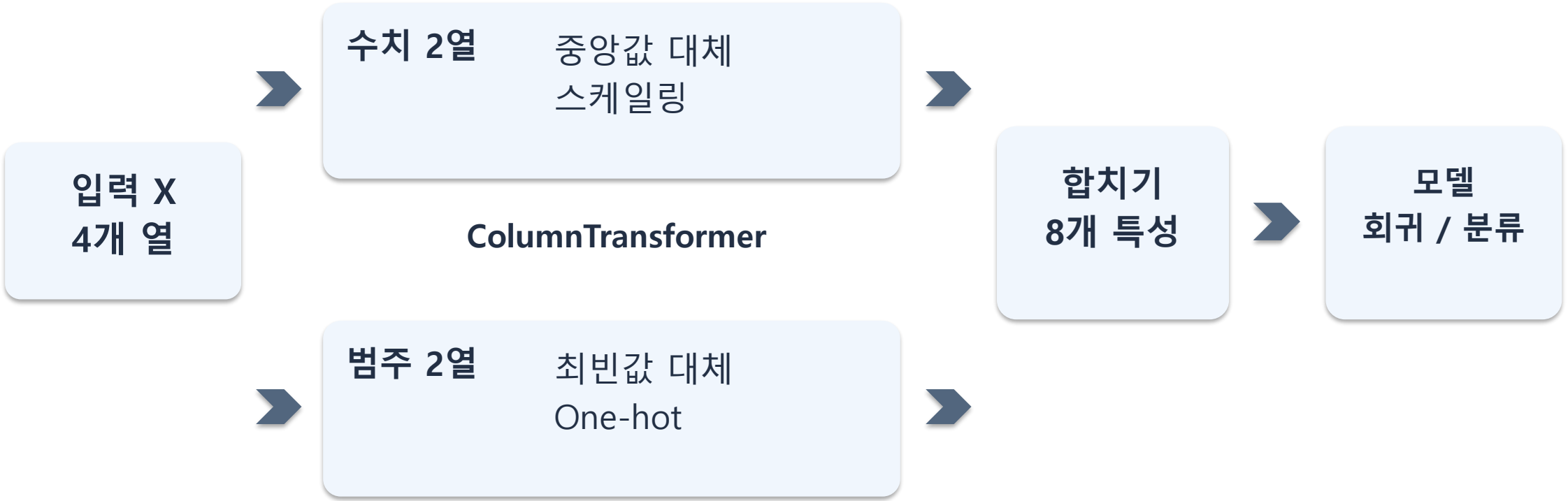
새 자료는 범위 밖일 수 있다.

실행 결과 · 요약

```
표준화: [-1.225, 0, 1.225]
MinMax: [0, 0.5, 1]
새 값 500 → 2.0
```

공식 참고: scikit-learn · StandardScaler / MinMaxScaler

수치형과 범주형은 각자의 경로로 처리



전처리와 모델을 Pipeline으로 묶어도 fit에 valid·test를 넣으면 누수다.

공식 참고: scikit-learn · ColumnTransformer / Pipeline

ColumnTransformer · 전날의 구조를 유지

```
num = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())])
cat = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore", sparse_output=False))])
pre = ColumnTransformer([
    ("num", num, NUM), ("cat", cat, CAT)])
train_z = pre.fit_transform(X_train)
valid_z = pre.transform(X_valid)
print("변환 shape:", train_z.shape, valid_z.shape)
```

E15 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

NUM / CAT
수치 2열 / 범주 2열 목록

sparse_output=False
교육용 작은 밀집 배열

handle_unknown="ignore"
새 범주는 해당 블록 0

실행 결과 · 요약

```
변환 shape: (60, 8) (20, 8)
```

공식 참고: scikit-learn · ColumnTransformer / Pipeline

model.fit과 predict 내부에서 일어나는 일

fit(train)

전처리 fit_transform
변환값으로 모델 fit

predict(valid)

전처리 transform
변환값으로 모델 predict

검사

중앙값·범주 수 불변
동일한 8열 구조

각 후보는 새로운 전처리·모델 객체로 생성한다. 학습된 객체를 다른 실험에 덮어쓰지 않는다.

공식 참고: scikit-learn · Pipeline / clone

스케일러만 바꾸어 공정하게 비교

```
for scale in ["none", "standard", "minmax"]:
    model = build(KNeighborsClassifier(n_neighbors=5), scale)
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    score = f1_score(y_valid, pred, zero_division=0)
    print(f"{scale}: F1={score:.3f}")
```

E16 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

고정 조건
같은 분할 · K=5 · p=2
weights="uniform"

변경 조건
num 경로의 스케일러만 변경

실행 결과 · 요약

```
none: F1=0.400
standard: F1=0.200
minmax: F1=0.400
```

공식 참고: scikit-learn · Nearest neighbors / Preprocessing

스케일링의 목적과 실험 결과를 구분한다

단위의 과도한 영향을 줄이는 것과 점수 상승을 보장하는 것은 다르다.

이 자료의 K=5 F1은 none 0.400 / standard 0.200 / minmax 0.400이다.

원핫 범주도 거리에 참여한다. 모든 열을 무조건 동일하게 변환하지 않는다.

유리한 결과만 남기지 말고, 바꾼 조건과 모든 후보 점수를 기록한다.

다음 교시의 K 실험은 standard를 고정한다. 그 결과를 전체 조합의 최적값이라고 말하지 않는다.

공식 참고: scikit-learn · Nearest neighbors / Preprocessing

실습 05 · 스케일링과 전처리 경로 비교

기본필수

E13·E14로 최근접 이웃과 새 값 범위를 확인한다.
공통 데이터의 전처리 결과 (60,8)·(20,8)을 재현한다.

응용

K=5를 고정하고 none·standard·minmax의 valid F1을 비교한다.
숫자 경로만 바뀌었는지와 train 통계 불변을 확인한다.

심화

수치·범주 특성의 거리 기여를 따로 살펴본다.
전체 배열을 표준화하는 대안은 별도 실험으로 표시하고 비교한다.

완료 기준: 변환 전후 예시 + shape·통계 검사 + 스케일링 비교 3행

3분 퀴즈 · 스케일과 누수

Q1. MinMaxScaler의 새 입력은 항상 0~1일까?

정답·해설 아니다. train 범위를 벗어난 새 값은 0~1 밖이 될 수 있다.

Q2. StandardScaler는 데이터를 정규분포로 만드는가?

정답·해설 아니다. 위치와 크기를 바꾸는 선형 변환이다.

Q3. Pipeline을 쓰면 test로 fit해도 괜찮을까?

정답·해설 아니다. 누수 방지는 객체 이름이 아니라 학습에 넣는 데이터의 문제다.

공식 참고: scikit-learn · MinMaxScaler / StandardScaler / Common pitfalls

학습되는 값과 직접 정하는 값을 구분한다

구분	예	어떻게 정하는가?
학습 파라미터	선형·로지스틱회귀의 계수와 절편	train에 fit하여 결정
하이퍼파라미터	K · weights · p · C	후보를 정하고 valid로 비교
전처리 선택	Standard / MinMax / 없음	실험 조건으로 명시하고 비교
판단 정책	실패 확률의 임계값	오류 비용과 valid를 보고 결정

KNN에는 회귀계수처럼 학습되는 w가 없지만, 학습 사례를 저장한 fitted 상태는 있다.

공식 참고: scikit-learn · Nearest neighbors / LogisticRegression / Decision threshold

실험 전에 고정 조건·후보·지표를 적는다

구분	분류 K 실험	회귀 K 실험
고정	같은 train60/valid20, X 4열	같은 train60/valid20, X 4열
전처리	중앙값·표준화 + 최빈값·원핫	동일
후보	K = 1, 3, 5, 9, 15	K = 1, 3, 5, 9, 15
나머지 설정	weights=uniform · p=2	weights=uniform · p=2
선택 규칙	valid F1 최대 · 동점이면 작은 K	valid RMSE 최소 · 동점이면 작은 K

후보 범위를 미리 정하고 결과를 모두 남긴다. 동일 valid를 많이 조회할수록 선택 편향이 생길 수 있다.

공식 참고: scikit-learn · Cross-validation

분류 · K 후보별 train과 valid F1 비교

```
for k in [1, 3, 5, 9, 15]:
    model = build(KNeighborsClassifier(n_neighbors=k))
    model.fit(X_train, y_train)
    tr_pred = model.predict(X_train)
    va_pred = model.predict(X_valid)
    tr = f1_score(y_train, tr_pred, zero_division=0)
    va = f1_score(y_valid, va_pred, zero_division=0)
    print(f"k={k:2d}: train={tr:.3f}, valid={va:.3f}")
```

E17 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

train F1
학습 자료에 얼마나 맞췄는가

valid F1
선택용 자료에서 어떤가

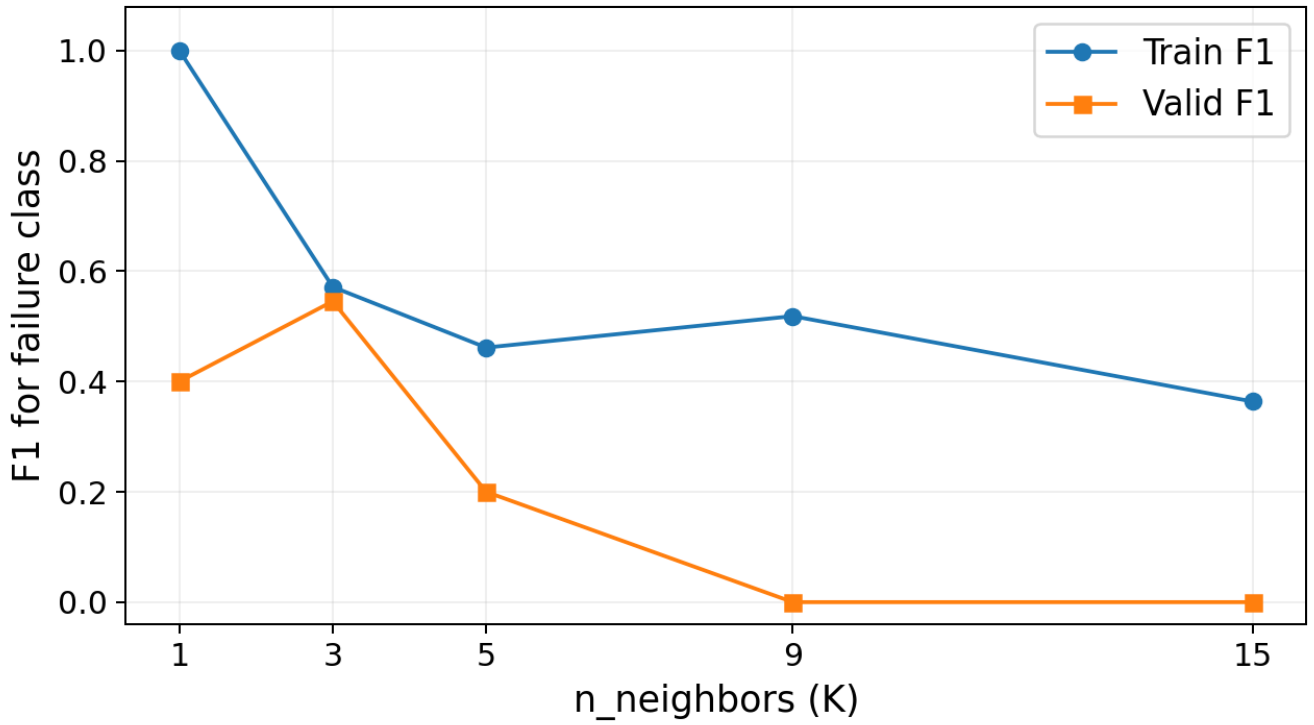
이번 후보의 선택: K=3

실행 결과 · 요약

K	train	valid
1	1.000	0.400
3	0.571	0.545
5	0.462	0.200
9	0.519	0.000
15	0.364	0.000

공식 참고: scikit-learn · KNeighborsClassifier / Cross-validation

K=1의 완벽한 train 점수가 답은 아니다



작은 K / 큰 K의 경향

작은 K: 개별 사례·잡음에 민감할 수 있다.

큰 K: 소수 클래스의 국소 패턴을 놓칠 수 있다.

실제 곡선은 매끄럽거나 단조롭지 않을 수 있다.

K=3은 이 고정 실험 안의 잠정 선택이다.

Day5 합성자료 · StandardScaler 고정 · train 60 / valid 20 · test 점수 없음

공식 참고: scikit-learn · Nearest neighbors / Cross-validation

회귀 · K는 목표에 맞춰 다시 비교한다

```
for k in [1, 3, 5, 9, 15]:
    model = build(KNeighborsRegressor(n_neighbors=k))
    model.fit(X_train, yr_train)
    pred = model.predict(X_valid)
    rmse = root_mean_squared_error(yr_valid, pred)
    print(f"k={k:2d}: valid RMSE={rmse:.2f} ms")
```

E18 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

회귀의 주 지표
valid RMSE · 단위 ms

작을수록 좋은 후보
이번 범위에서는 K=15

분류의 K=3을 복사하지 않음

실행 결과 · 요약

```
K   valid RMSE (ms)
1    944.68
3    795.22
5    744.99
9    707.65
15   641.79
```

공식 참고: scikit-learn · KNeighborsRegressor / root_mean_squared_error

과적합을 진단할 때 확인할 것

의심 신호

train은 좋은데 valid는 크게 나쁘다.

설정을 조금 바꾸면 valid 점수가 크게 흔들린다.

학습 사례에 맞춘 설명만 있고 새 입력 검증이 없다.

단정하기 전에

데이터 누수·라벨·분할의 문제를 먼저 확인한다.

낮은 점수가 곧 과적합인 것은 아니다.

신호 부족·분포 차이·표본 수의 영향도 검토한다.

valid 20건에서 1건 차이는 Accuracy 0.05다. 작은 차이에 확정적 의미를 부여하지 않는다.

공식 참고: scikit-learn · Cross-validation / Common pitfalls

재현성과 공정성 · seed만 적으면 끝이 아니다

원본 데이터 버전과 행 순서, 실제 분할 ID를 함께 보관한다.

전처리와 모델의 모든 변경 조건을 비교표에 적는다.

train 점수·valid 점수·오류 수를 함께 기록한다.

test는 보관한다. 최종 모델·기준을 확정한 뒤 한 번 평가한다.

공식 참고: scikit-learn · train_test_split / Cross-validation

실습 06 · 하이퍼파라미터 실험표

기본필수	standard를 고정한 K=1·3·5·9·15의 분류 F1 표를 만든다. K=3을 고른 근거와 K=1의 train=1.000을 설명한다.
응용	같은 후보로 회귀 RMSE 표를 추가한다. 분류 K=3, 회귀 K=15가 서로 다른 이유를 정답·지표 관점으로 설명한다.
심화	C 또는 weights 중 하나만 추가로 비교한다. 후보 수를 제한하고 모든 결과·동점 규칙·valid 반복 조회의 한계를 기록한다.

완료 기준: 고정 조건 + 후보별 표 + 선택 규칙 + 일반화에 대한 한계

3분 퀴즈 · 비교 실험의 규칙

Q1. K를 바꿀 때 split도 바꾸면 무엇이 문제인가?

정답·해설 모델 설정의 효과와 데이터 구성의 효과를 분리할 수 없다.

Q2. 회귀와 분류에서 같은 K를 써야 공정할까?

정답·해설 아니다. 같은 문제 안에서 조건을 맞추고, 각 목표의 지표로 따로 고른다.

Q3. 지금의 최고 valid 점수를 최종 성능이라고 보고해도 될까?

정답·해설 후보 선택에 사용한 점수다. 선택 편향이 있으므로 최종 test와 구분한다.

공식 참고: scikit-learn · Cross-validation

같은 데이터·다른 목표 · 비교표는 두 개

공통 입력·분할

X 4열 · train60/valid20
같은 전처리 조건

회귀 비교

Dummy · Linear · KNN15
MAE · RMSE · R^2

분류 비교

Dummy · Logistic · KNN3
F1 · Precision · Recall

회귀의 RMSE 640과 분류의 F1 0.545는 서로 비교하거나 평균낼 수 있는 값이 아니다.

분류 비교표 · 후보마다 독립 Pipeline을 만든다

```
candidates = {
    "Dummy": DummyClassifier(strategy="most_frequent"),
    "Logistic": LogisticRegression(C=1.0, max_iter=1000),
    "KNN_k3": KNeighborsClassifier(n_neighbors=3)}
rows = []
for name, estimator in candidates.items():
    model = build(estimator).fit(X_train, y_train)
    pred = model.predict(X_valid)
    rows.append({"model": name, **cls_metrics(y_valid, pred)})
cls_table = pd.DataFrame(rows)
print(cls_table[["model", "F1", "FP", "FN"]].round(3))
```

E20 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

candidates.items()
이름과 모델을 하나씩 선택

cls_metrics()
F1·오류 수를 계산하는 함수

새로운 build()
전처리 객체도 새로 생성

실행 결과 · 요약

```
F1 / FP / FN
Dummy    0.000 / 0 / 6
Logistic 0.333 / 4 / 4
KNN3     0.545 / 2 / 3
```

공식 참고: scikit-learn · Pipeline / f1_score

분류 비교 결과 · F1과 놓친 실패 수를 함께

모델	Accuracy	Precision	Recall	F1	FP / FN
Dummy	0.700	0.000	0.000	0.000	0 / 6
Logistic · C=1	0.600	0.333	0.333	0.333	4 / 4
KNN · K=3	0.750	0.600	0.500	0.545	2 / 3

잠정 선택: KNN3. 그래도 실제 실패 6건 중 3건을 놓친다. 운영 사용을 승인하는 결과가 아니다.

공식 참고: scikit-learn · f1_score / confusion_matrix

회귀 비교표 · 기준선도 같은 조건으로 평가

```
candidates = {
    "Dummy": DummyRegressor(strategy="mean"),
    "Linear": LinearRegression(),
    "KNN_k15": KNeighborsRegressor(n_neighbors=15)}
rows = []
for name, estimator in candidates.items():
    model = build(estimator).fit(X_train, yr_train)
    metrics = reg_metrics(yr_valid, model.predict(X_valid))
    rows.append({"model": name, **metrics})
reg_table = pd.DataFrame(rows)
print(reg_table.round(2).to_string(index=False))
```

E21 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

DummyRegressor(mean)
train y 평균만 예측

reg_metrics()
MAE·RMSE·R^2 계산

RMSE 기준으로
Linear를 잠정 선택

실행 결과 · 요약

```
RMSE (ms)
Dummy:   658.75
Linear:  640.00
KNN15:   641.79
```

공식 참고: scikit-learn · mean_absolute_error / root_mean_squared_error / r2_score

회귀 비교 결과 · 1.79ms 차이를 과장하지 않기

모델	MAE (ms)	RMSE (ms)	R^2
Dummy · train 평균	541.26	658.75	−0.162
LinearRegression	551.80	640.00	−0.097
KNN · K=15	530.06	641.79	−0.103

RMSE는 Linear, MAE는 KNN15가 작다. 주 지표를 먼저 정한 이유가 여기서 드러난다.

공식 참고: scikit-learn · mean_absolute_error / root_mean_squared_error / r2_score

선택 근거는 점수 하나가 아니라 판단 기록이다

질문	분류 예시	회귀 예시
무엇을 줄이려는가?	실패 놓침과 오탐을 함께 본다.	큰 소요시간 오차를 더 줄인다.
사전 주 지표는?	실패 클래스 F1	RMSE · ms
어떤 후보를 골랐나?	KNN3 · F1=0.545	Linear · RMSE=640.00
남은 문제는?	실패 6건 중 3건을 놓침	MAE는 KNN15가 더 낮음
어떻게 말할 것인가?	이 valid의 잠정 선택	작은 차이 · 낮은 설명력 · 추가 검증

F1이 실제 비용을 자동 최적화하지는 않는다. 오류 비용을 수치화하는 심화는 10/28로 연결한다.

공식 참고: scikit-learn · f1_score / Cross-validation

실습 프로젝트 · 책임과 결과 파일을 나눈다

```
day6_compare/
  main.py
  preprocessing.py
  evaluate.py
  data/requests.csv
  data/split_manifest.csv
  outputs/
    regression_comparison.csv
    classification_comparison.csv
    knn_search.csv
    validation_predictions.csv
  reports/model_choice.md
  README.md
  requirements-lock.txt
```

역할과 출처

main: 읽기·같은 ID 선택·후보 실행

preprocessing: 열별 변환과 모델 조합

evaluate: 문제별 지표·오류 수 계산

data: Day5 원본·분할표 재사용

reports: 지표 선택 이유·한계

이번 슬라이드는 실습 시 만들 구조다. PPT 파일과 별도로 실습 패키지가 이미 배포되었다는 뜻은 아니다.

60분 통합 · 만든 코드를 이어서 재현한다

입력·학습

ID·열 계약 확인
train-only 전처리·fit

비교·저장

회귀·분류 각 3후보
비교표·검증 예측 저장

설명·검사

지표 선택 근거
test 보관·Git 기록

7교시 실습 25분 + 8교시 실습 35분 · 새 기능보다 재실행 가능한 완료를 우선한다.

자동 검사 · predict가 학습 통계를 바꾸지 않는가?

```
model = build(KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)
imp = model["pre"].named_transformers_["num"]["imputer"]
before = imp.statistics_.copy()
model.predict(X_valid)
after = imp.statistics_
print("통계 불변:", np.array_equal(before, after))
print("입력 열 일치:", list(X_train.columns) == FEATURES)
print("금지 열 없음:", not set(FORBIDDEN) & set(X_train.columns))
```

E23 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

statistics_.copy()
중앙값을 별도로 보관

array_equal()
예측 전후 값이 같은지 확인

입력 열 목록도 함께 검사

실행 결과 · 요약

```
통계 불변: True
입력 열 일치: True
금지 열 없음: True
```

공식 참고: scikit-learn · Common pitfalls / Pipeline

선택 Pipeline을 저장하고 예측을 대조

```
from pathlib import Path
import tempfile, joblib
model = build(KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)
with tempfile.TemporaryDirectory() as folder:
    path = Path(folder) / "selected_pipeline.joblib"
    joblib.dump(model, path)
    restored = joblib.load(path)
    before = model.predict(X_valid)
    after = restored.predict(X_valid)
    print("복원 예측 동일:", np.array_equal(before, after))
```

E24 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

joblib.dump()
전처리와 모델을 함께 저장

joblib.load()
본인이 만든 파일만 읽기

같은 환경에서 전후 예측 대조

실행 결과 · 요약

```
복원 예측 동일: True
```

공식 참고: scikit-learn · Model persistence

실습 07 · 회귀·분류 비교 프로젝트 통합

기본필수

전날 원본·분할 ID를 재사용한다. 회귀 3후보와 분류 3후보를 학습하고 두 비교표와 valid 예측 결과를 저장한다.

응용

모듈을 분리하고 K 탐색표와 Pipeline 저장·복원을 연결한다. 짝이 README만 보고 같은 결과를 만들게 한다.

심화

미지 범주·필수 열 누락·과도한 K·학습 통계 불변 중 두 가지 이상을 자동 검사하고 실패 원인을 명시한다.

완료 기준: 실행 가능한 초안 + 두 비교표 + valid 예측 + test 미사용 기록

최종 제출 · 수치와 판단 근거를 함께 남긴다

산출물	필수 내용
regression_comparison.csv	Dummy·Linear·KNN15 / MAE·RMSE·R^2 / 조건
classification_comparison.csv	Dummy·Logistic·KNN3 / F1·Precision·Recall·FP·FN
knn_search.csv	문제 유형 · K · 스케일링 · train/valid 지표
validation_predictions.csv	request_id · 문제 · 모델 · 실제값 · 예측값
reports/model_choice.md	선택 지표 · 선정 이유 · 오류 사례 · 한계
README / Git 이력	실행 방법 · 버전 · 분할 출처 · 변경 이력

test 점수 열은 만들지 않는다. “미평가”는 누락이 아니라 오늘의 평가 정책이다.

보고서 · 다섯 문장으로 선택을 설명한다

목표: 실행 전에 실패 위험과 예상 소요시간을 예측한다.

조건: 같은 train60/valid20, 입력4열, train-only 전처리를 사용했다.

선택: 분류는 F1 기준 KNN3, 회귀는 RMSE 기준 Linear를 잠정 선택했다.

한계: 실패 3건을 놓쳤고, 회귀 RMSE 차이는 1.79ms에 불과했다.

다섯 번째 문장: 합성 자료·소표본의 한계를 밝히고, test 보관과 다음 검증 계획을 적는다.

정상·경계·실패 입력으로 확인한다

상황	기대 결과	확인 방법
valid 20건	각 모델 예측 20개 · ID 순서 일치	길이·ID·실제값 정렬 검사
수치 결측·새 범주	학습된 규칙으로 8개 특성 유지	shape·NaN/inf·통계 검사
필수 열 누락	학습/예측 전 명확한 오류	열 이름 안내 · 임의 값 생성 금지
K > train 행 수	모델 실행 전 설정 오류 확인	1 ≤ K ≤ 60 검사
자체 저장 모델 복원	동일 환경에서 같은 valid 예측	저장 전후 배열 대조

“에러가 나지 않았다”만으로 통과하지 않는다. 기대 출력·단위·행 수와 정책을 먼저 정한다.

공식 참고: scikit-learn · Pipeline / KNeighborsClassifier

형성평가 · 비교 구조와 설명을 평가한다

항목	배점	충족 기준
데이터·누수 방지	20	같은 ID·허용 열·train-only fit·test 보관
모델 구현	25	회귀·분류 모델과 기준선을 Pipeline으로 실행
지표·비교표	25	문제별 지표·단위·오류 수·설정 기록
선택 근거·한계	20	사전 주 지표·선택 이유·오류 사례·소표본 한계
재현·협업 기록	10	README·버전·결과 재생성·Git 변경 이력

기본필수만으로 100점 기준을 충족할 수 있다. 과목 최종 실습평가는 10/28에 진행한다.

실습 08 · 최종 검증·시연·지표 선택 근거

기본필수

한 번의 실행으로 두 비교표와 예측 결과를 다시 만든다.
주 지표·선택 이유·오류 사례·test 보관을 본인의 말로 설명한다.

응용

짝이 실행을 재현하고 저장 모델의 예측을 대조한다.
같은 데이터의 회귀/분류 선택 기준이 다른 이유를 발표한다.

심화

정상·경계·실패 테스트를 시연한다. 다음 트리·앙상블과
비교해도 유지할 분할·데이터 계약·평가 정책을 정리한다.

완료 기준: 코드 + 두 비교표 + 예측·탐색 기록 + 보고서 + Git 이력

마무리 3분 확인 퀴즈

Q1. 회귀 RMSE와 분류 F1을 평균내도 되는가?

정답·해설 안 된다. 목표·단위·정의가 다른 지표다. 비교표를 따로 만든다.

Q2. KNN3이 최고 valid F1이면 곧바로 배포해도 될까?

정답·해설 아니다. 실패 절반을 놓치고 소표본·선택 편향이 남아 있다.

Q3. 내일 모델을 트리로 바꾸어도 그대로 남길 것은?

정답·해설 원본·입력 계약·분할 ID·선택 지표·test 보관 정책이다.

공식 참고: scikit-learn · Cross-validation / f1_score

오늘의 핵심 정리

원리: 선형회귀는 가중합, 로지스틱회귀는 확률, KNN은 이웃을 이용한다.

비교: 분할·입력·전처리·지표를 명시하고 바꾼 조건만 비교한다.

선택: 회귀와 분류의 지표를 분리하고 오류 비용과 한계를 함께 설명한다.

오늘 남길 것: 회귀 비교표 + 분류 비교표 + 근거 있는 선택 기록 + 재현 가능한 코드

다음 수업 · 같은 기준으로 더 복잡한 모델 비교

일자	다음 내용	오늘과 연결
10/27 (화)	Decision Tree · Random Forest · XGBoost	같은 분할로 과적합·특성 중요도 비교
10/28 (수)	지표 심화 · 교차검증 · 최종 Pipeline	선택 정책 확정 후 최종 test 평가
10/29 (목)	GPU · Tensor · ANN · 역전파	학습·추론·손실의 구분 재사용
10/30 (금)	Optimizer · 학습률 · 과적합 제어	train/valid 차이를 학습곡선으로 진단

모델을 바꾸기 전에 데이터를 다시 나누지 않는다. 오늘의 비교표를 다음 실험의 기준으로 보관한다.

공식 참고자료 · 모델과 평가

선형·로지스틱회귀	LinearRegression · Linear models · LogisticRegression · log_loss
KNN 원리·설정	Nearest neighbors · KNeighborsClassifier · KNeighborsRegressor
회귀 지표	mean_absolute_error · root_mean_squared_error · r2_score
분류 지표·임계값	confusion_matrix · precision_score · recall_score · f1_score · Decision threshold

공식 문서 확인: 2026-10-21 · stable 문서는 갱신될 수 있음 · 실제 실행 검증 버전은 6쪽에 명시

공식 참고자료 · 전처리·검증·재현

스케일링	StandardScaler · MinMaxScaler · Normalizer · Preprocessing
전처리·모델 연결	ColumnTransformer · Pipeline · clone
데이터 분리·누수 방지	Common pitfalls · Cross-validation · train_test_split
기준선·저장	DummyRegressor · DummyClassifier · Model persistence

공식 문서 확인: 2026-10-21 · stable 문서는 갱신될 수 있음 · 실제 실행 검증 버전은 6쪽에 명시